Universidad Tecnológica de Durango

A08 · Diseño funcional: algoritmos, diagramas de flujo y casos de uso

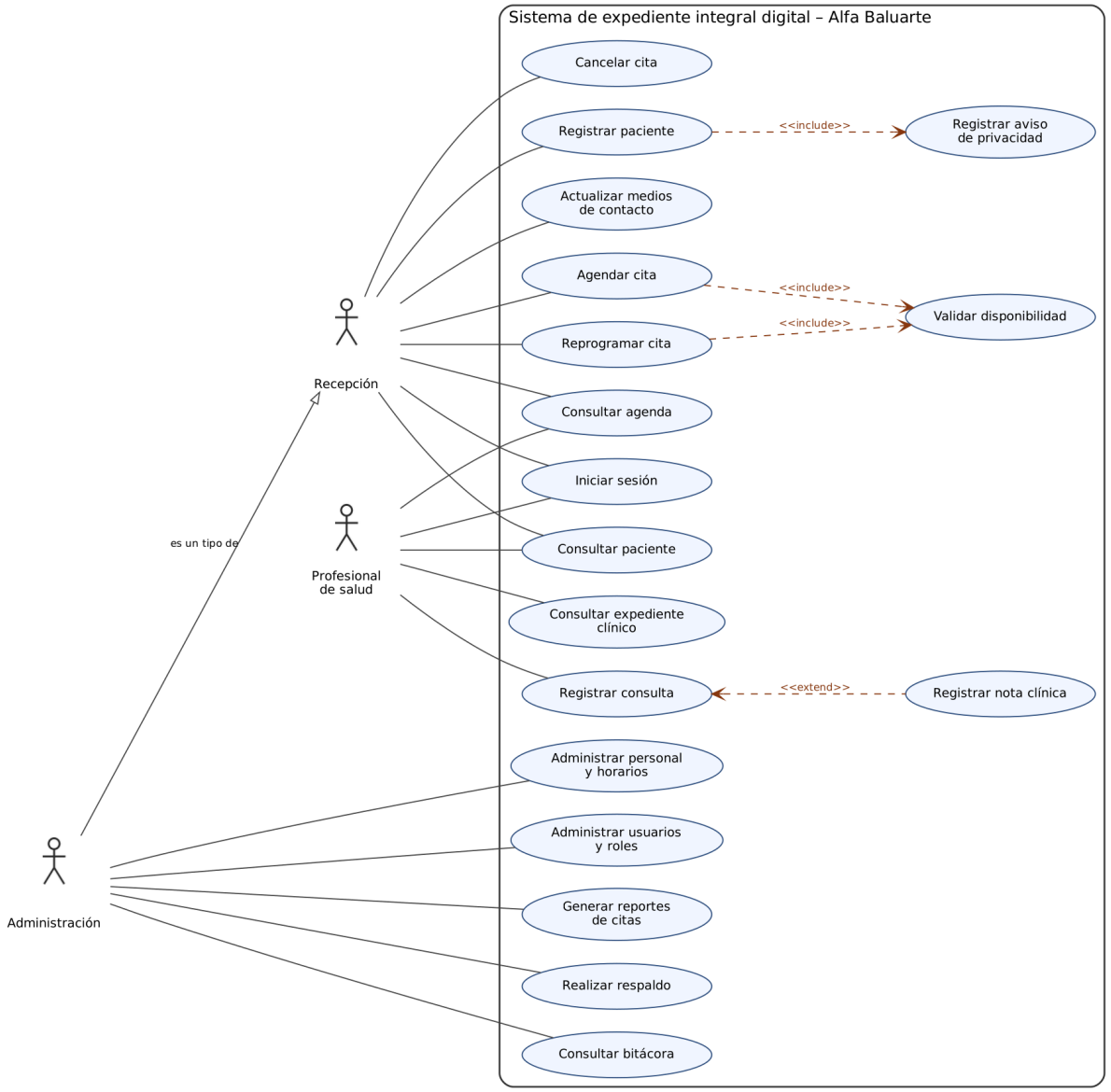
Proyecto Alfa Baluarte: sistema de escritorio local para la clínica (Victoria de Durango)

Datos del documento

Campo	Dato
Asignatura	Proyecto Integrador I (E-PIN1-1)
Unidad	III. Diseño, aplicación, desarrollo y pruebas del proyecto
Actividad	A08 · Diseño funcional: algoritmos, diagramas de flujo y casos de uso
Profesor	Raúl Iván Herrera González
Grupo	4B BIS TI · Universidad Tecnológica de Durango
Equipo	Alfa Baluarte (Equipo 2)
Integrantes	Dylan Percy Rodriguez Sanchez Miguel Isai Escajeda Sariñana Karol Emmanuel Guerrero Quintero (Product Ownerr), Luis Alejandro Macias Salas Leonel Ivan Sifuentes Zaragoza
Proyecto	Sistema de escritorio para la clínica de Alfa Baluarte: expediente integral digital de pacientes y personal
Archivo a entregar	Equipo02_A08_AlfaBaluarte.pdf (en /portafolio/A08/)
Fuentes de diagramas	/docs/diseno/ del repositorio (.drawio, .puml o .psc)
Fecha de entrega	6 / 11 / 2026

1. Diagrama de casos de uso

[Imagen del diagrama de casos de uso]



Actor	Descripción	Casos de uso
Recepción	Atiende a los pacientes y maneja la agenda	Iniciar sesión, registrar paciente, actualizar contacto, consultar paciente, agendar, reprogramar y cancelar cita, consultar agenda
Administración	Gestiona el sistema y también hace las tareas de recepción	Administrar personal y horarios, administrar usuarios y roles, generar reportes, realizar respaldo, consultar bitácora, más todo lo de Recepción
Profesional de salud	Atiende a los pacientes y registra el expediente	Iniciar sesión, consultar paciente, consultar agenda, consultar expediente, registrar consulta y nota clínica

2. Especificación de casos de uso

Especifica de 3 a 5 casos prioritarios, los ligados a historias Must. Copia la ficha para CU-04 y CU-05 si los necesitas. Cada ficha debe tener al menos un flujo alterno o de error.

CU-01 · (nombre del caso de uso)

Campo	Contenido
ID y nombre	CU-01 · Registrar Paciente
Actor(es)	Recepcion
Requisito / historia	RF-__ / HU-__
Precondiciones	El usuario inicio sesion con rol de recepcion o administracion.
Flujo básico	1. El usuario abre el módulo Pacientes y elige “Nuevo paciente”. 2. Captura nombre, primer apellido, segundo apellido

	(opcional), fecha de nacimiento y sexo. 3. Captura teléfono y, si hay, correo. 4. El sistema verifica que el paciente no exista. 5. El usuario registra la aceptación del aviso de privacidad. 6. El sistema guarda al paciente y le asigna su folio. 7. El sistema muestra “Paciente registrado” con el folio.
Flujos alternos / de error	4a. El paciente ya existe: el sistema avisa y muestra el registro existente. 3a. El teléfono no tiene 10 dígitos: el sistema lo rechaza y pide corregirlo. 5a. El paciente no acepta el aviso: el registro no se completa.
Postcondiciones	El paciente queda activo con folio, contacto y consentimiento, y el movimiento queda en la bitácora.
Reglas de negocio	RN-01, RN-02, RN-03

CU-02 · (nombre del caso de uso)

Campo	Contenido
ID y nombre	CU-02 · Agendar cita
Actor(es)	Recepcion
Requisito / historia	RF-__ / HU-__
Precondiciones	Sesión iniciada; el paciente está registrado y activo; el profesional tiene horario registrado.
Flujo básico	1. El usuario busca al paciente. 2. Elige el servicio. 3. El sistema muestra los profesionales que dan ese servicio. 4. El usuario elige profesional, fecha y hora. 5. El sistema valida la disponibilidad. 6. El usuario elige consultorio y modalidad. 7. El sistema guarda la cita en estado “pendiente” con su folio.

Flujos alternos / de error	5a. El horario está ocupado o fuera del horario del profesional: el sistema lo rechaza y ofrece horarios libres. 1a. El paciente está dado de baja: no se puede agendar. 4a. El profesional no da ese servicio: el sistema no lo ofrece.
Postcondiciones	La cita existe en estado “pendiente” y aparece en la agenda del profesional.
Reglas de negocio	RN-04, RN-05, RN-06, RN-07

CU-03 · (nombre del caso de uso)

Campo	Contenido
ID y nombre	CU-03 · Registrar consulta y nota clínica
Actor(es)	Profesional de salud
Requisito / historia	RF-__ / HU-__
Precondiciones	Sesión iniciada con rol profesional; existe una cita atendida o por atender con ese profesional.
Flujo básico	1. El profesional abre su agenda y elige la cita. 2. Abre el expediente del paciente. 3. Captura motivo de consulta y diagnóstico. 4. Escribe una o más notas clínicas. 5. El sistema guarda la consulta y las notas, y marca la cita como completada.
Flujos alternos / de error	1a. La cita está cancelada: no se puede registrar consulta. 4a. Si hay un error en una nota guardada, no se edita: se registra una nota nueva que la corrige.
Postcondiciones	La consulta y las notas quedan en el expediente, firmadas por el profesional, y el movimiento queda en la bitácora.

Reglas de negocio	RN-08, RN-09

3. Reglas de negocio

Lista las reglas que aparecen en tus casos de uso: validaciones, cálculos, restricciones y permisos.

ID	Regla	Caso de uso	Tipo (validación / cálculo / restricción / permiso)
RN-01	No puede haber dos pacientes con el mismo nombre, apellidos y fecha de nacimiento.	CU-01	Restricción
RN-02	No puede haber dos pacientes con el mismo nombre, apellidos y fecha de nacimiento.	CU-01	Validación
RN-03	El teléfono debe tener 10 dígitos. Un cambio de teléfono o correo crea un registro nuevo y deja inactivo el anterior.	CU-01	Validación
RN-04	No se puede agendar a un paciente ni a un profesional dados de baja.	CU-02	Restricción
RN-05	El profesional debe ofrecer el servicio elegido para la cita.	CU-02	Validación
RN-06	La cita debe caer dentro del horario de atención del profesional.	CU-02	Validación
RN-07	El profesional, el paciente y el consultorio no pueden tener dos citas al mismo tiempo.	CU-02	Restricción
RN-08	Solo el personal clínico puede registrar consultas y notas, y	CU-03	Permiso

	solo de citas que no estén canceladas.		
RN-09	Las notas clínicas no se editan. Una corrección se guarda como una nota nueva que apunta a la anterior.	CU-03	Restricción

4. Algoritmos en pseudocódigo

A continuación se presentan tres algoritmos de la lógica de negocio del sistema, uno por cada caso de uso prioritario. Cada algoritmo usa decisiones (Si… Entonces) y repeticiones (Mientras). El comentario // RN-__ indica la regla de negocio que se aplica en ese paso.

Algoritmo 1 · Registrar paciente

Campo	Contenido
Nombre	RegistrarPaciente
Propósito	Dar de alta a un paciente nuevo validando sus datos, que no esté duplicado y que haya aceptado el aviso de privacidad.
Entradas	nombre, primerApellido, segundoApellido (opcional), fechaNacimiento, sexo, telefono, aceptaAviso
Salidas	Mensaje de éxito con el folio del paciente, o mensaje de error.
Caso de uso / regla	CU-01 / RN-01, RN-02, RN-03

Pseudocódigo

```
Inicio RegistrarPaciente(nombre, primerApellido, segundoApellido,

fechaNacimiento, sexo, telefono, aceptaAviso)

Si nombre = "" O primerApellido = "" O fechaNacimiento > fechaActual Entonces
```

```
Mostrar "Datos obligatorios incompletos o fecha inválida"; Terminar

FinSi

Si longitud(telefono) <> 10 Entonces

Mostrar "El teléfono debe tener 10 dígitos"; Terminar // RN-03

FinSi

i ← 1

Mientras i <= 10 Hacer

Si NO esDigito(telefono[i]) Entonces

Mostrar "El teléfono solo puede tener números"; Terminar // RN-03

FinSi

i ← i + 1

FinMientras

Si existePaciente(nombre, primerApellido, segundoApellido, fechaNacimiento) Entonces

Mostrar "El paciente ya está registrado"; Terminar // RN-01

FinSi

Si aceptaAviso = falso Entonces

Mostrar "Debe aceptar el aviso de privacidad"; Terminar // RN-02

FinSi

folio ← generarFolio()

idPaciente ← guardarPaciente(folio, nombre, primerApellido,

segundoApellido, fechaNacimiento, sexo)

guardarContacto(idPaciente, "telefono", telefono, esPrincipal ← verdadero)

guardarConsentimiento(idPaciente, versionAvisoVigente, fechaActual)

registrarBitacora("alta", "pacientes", idPaciente)

Mostrar "Paciente registrado. Folio: ", folio

Fin
```

Algoritmo 2 · Agendar cita

Campo	Contenido

Nombre	AgendarCita
Propósito	Crear una cita comprobando que paciente y profesional estén activos, que el profesional dé el servicio, que esté dentro de su horario y que no haya empalmes.
Entradas	idPaciente, idProfesional, idServicio, fecha, horaInicio, consultorio, modalidad
Salidas	Cita en estado “pendiente” con su folio, o mensaje de error.
Caso de uso / regla	CU-02 / RN-04, RN-05, RN-06, RN-07

Pseudocódigo

```
Inicio AgendarCita(idPaciente, idProfesional, idServicio, fecha,

horaInicio, consultorio, modalidad)

Si estatusPaciente(idPaciente) = "baja" O

estatusPersonal(idProfesional) = "baja" Entonces

Mostrar "Paciente o profesional dado de baja"; Terminar // RN-04

FinSi

Si NO profesionalDaServicio(idProfesional, idServicio) Entonces

Mostrar "El profesional no ofrece ese servicio"; Terminar // RN-05

FinSi

horaFin ← horaInicio + duracionServicio(idServicio)

Si NO dentroDeHorario(idProfesional, fecha, horaInicio, horaFin) Entonces

Mostrar "Fuera del horario del profesional"; Terminar // RN-06

FinSi

citas ← citasActivasDelDia(fecha)

hayEmpalme ← falso

i ← 1

Mientras i <= totalCitas(citas) Y hayEmpalme = falso Hacer

mismoRecurso ← comparteRecurso(citas[i], idProfesional, idPaciente, consultorio)

Si mismoRecurso Y horaInicio < citas[i].horaFin Y
```

```
horaFin > citas[i].horaInicio Entonces

hayEmpalme ← verdadero

FinSi

i ← i + 1

FinMientras

Si hayEmpalme Entonces

Mostrar "Ese horario ya está ocupado"; Terminar // RN-07

FinSi

folio ← generarFolioCita()

idCita ← guardarCita(folio, idPaciente, idProfesional, idServicio, fecha,

horaInicio, horaFin, consultorio, modalidad, "pendiente")

registrarBitacora("alta", "citas", idCita)

Mostrar "Cita agendada. Folio: ", folio

Fin
```

Algoritmo 3 · Registrar consulta y notas clínicas

Campo	Contenido
Nombre	RegistrarConsulta
Propósito	Guardar la consulta de una cita y sus notas clínicas, sin editar notas anteriores.
Entradas	idCita, idProfesional, motivoConsulta, diagnostico, listaNotas (cada nota con su texto y, si corrige otra, el id de la nota original)
Salidas	Consulta y notas guardadas, cita marcada como completada, o mensaje de error.
Caso de uso / regla	CU-03 / RN-08, RN-09

Pseudocódigo

```
Inicio RegistrarConsulta(idCita, idProfesional, motivoConsulta,

diagnostico, listaNotas)

Si NO esPersonalClinico(idProfesional) Entonces

Mostrar "No tiene permiso para registrar consultas"; Terminar // RN-08

FinSi

Si estadoCita(idCita) = "cancelada" Entonces

Mostrar "La cita está cancelada"; Terminar // RN-08

FinSi

Si motivoConsulta = "" Entonces

Mostrar "El motivo de consulta es obligatorio"; Terminar

FinSi

idConsulta ← guardarConsulta(idCita, motivoConsulta, diagnostico)

n ← 1

Mientras n <= totalNotas(listaNotas) Hacer

Si listaNotas[n].corrige <> vacio Entonces

guardarNota(idConsulta, idProfesional,

listaNotas[n].texto, listaNotas[n].corrige) // RN-09

Si no

guardarNota(idConsulta, idProfesional, listaNotas[n].texto, vacio)

FinSi

n ← n + 1

FinMientras

cambiarEstadoCita(idCita, "completada")

registrarBitacora("alta", "consultas", idConsulta)

Mostrar "Consulta registrada"

Fin
```

5. Pruebas de escritorio

Algoritmo 1 · Caso válido (paciente nuevo)

Datos de entrada: nombre="Ana",

primerApellido="López",

segundoApellido="Ruiz",

fechaNacimiento=2000-05-14,

sexo="F",

telefono="6181234567",

aceptaAviso=verdadero.

Supuestos:

fechaActual=2026-11-05 y el paciente no existe en la base.

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	nombre="Ana", primerApellido="López", fechaNacimiento=2000-05-14, fechaActual=2026-11-05	nombre="" O primerApellido="" O fechaNacimiento > fechaActual	Falso	Continúa
2	telefono="6181234567", longitud(telefono)=10	longitud(telefono) <> 10 (RN-03)	Falso	Continúa
3	i=1, telefono[1]="6"	i <= 10 y NO esDigito("6")	Verdadero y Falso	i ← 2
4	i=2, telefono[2]="1"	i <= 10 y NO esDigito("1")	Verdadero y Falso	i ← 3
5	i=3, telefono[3]="8"	i <= 10 y NO esDigito("8")	Verdadero y Falso	i ← 4
6	i=4, telefono[4]="1"	i <= 10 y NO esDigito("1")	Verdadero y Falso	i ← 5
7	i=5, telefono[5]="2"	i <= 10 y NO esDigito("2")	Verdadero y Falso	i ← 6
8	i=6, telefono[6]="3"	i <= 10 y NO esDigito("3")	Verdadero y Falso	i ← 7
9	i=7, telefono[7]="4"	i <= 10 y NO esDigito("4")	Verdadero y Falso	i ← 8
10	i=8, telefono[8]="5"	i <= 10 y NO esDigito("5")	Verdadero y Falso	i ← 9
11	i=9, telefono[9]="6"	i <= 10 y NO esDigito("6")	Verdadero y Falso	i ← 10
12	i=10, telefono[10]="7"	i <= 10 y NO esDigito("7")	Verdadero y Falso	i ← 11
13	i=11	i <= 10	Falso	Sale del ciclo (todos son dígitos)

14	**existePaciente("Ana","López","Ruiz", 2000-05-14)=falso**	**existePaciente(...) (RN-01)**	**Falso**	**Continúa**
15	**aceptaAviso=verdadero**	**aceptaAviso = falso (RN-02)**	**Falso**	**Continúa**
16	**folio="PAC-000009", idPaciente=9**	**(sin condición: se guardan paciente, contacto principal, consentimiento y bitácora)**	**—**	**"Paciente registrado. Folio: PAC-000009"**

Resultado: el paciente queda registrado con folio, teléfono principal y consentimiento. Cumple RN-01, RN-02 y RN-03.

Algoritmo 1 · Caso de error (teléfono incompleto)

Datos de entrada:

nombre="Ana",

primerApellido="López",

segundoApellido="Ruiz",

fechaNacimiento=2000-05-14,

sexo="F",

telefono="618123456",

aceptaAviso=verdadero.

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	**nombre="Ana", primerApellido="López", fechaNacimiento=2000-05-14, fechaActual=2026-11-05**	**nombre="" O primerApellido="" O fechaNacimiento > fechaActual**	**Falso**	**Continúa**
2	**telefono="618123456", longitud(telefono)=9**	**longitud(telefono) <> 10 (RN-03)**	**Verdadero**	**"El teléfono debe tener 10 dígitos"; Terminar**

Resultado: no se guarda nada y el algoritmo termina. Se aplica RN-03.

Algoritmo 2 · Caso válido (cita sin empalme)

Datos de entrada:

idPaciente=9 (activo),

idProfesional=3 (activo),

idServicio=1,

fecha=2026-11-09.,

horaInicio=10:00,

consultorio="Consultorio 1",

modalidad="Presencial".

Horario del profesional 3 los lunes: 09:00 a 14:00.

Citas existentes ese día: [1]

profesional 4,

paciente 2,

"Consultorio 2",

10:00-11:00;

[2] profesional 3, paciente 5, "Consultorio 1", 12:00-13:00.

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	estatusPaciente(9)="activo", estatusPersonal(3)="activo"	estatusPaciente = "baja" O estatusPersonal = "baja" (RN-04)	Falso	Continúa
2	profesionalDaServicio(3,1)=verdadero	NO profesionalDaServicio(3,1) (RN-05)	Falso	Continúa
3	duracionServicio(1)=60 min, horaInicio=10:00	horaFin ← horaInicio + duración	—	horaFin = 11:00
4	dentroDeHorario(3, 2026-11-09, 10:00, 11:00): el horario es 09:00-14:00	NO dentroDeHorario(...) (RN-06)	Falso	Continúa
5	citas = 2 citas del día; hayEmpalme=falso; i=1	—	—	Inicia el ciclo
6	i=1, citas[1]: profesional 4, paciente 2, Consultorio 2; mismoRecurso=falso	i <= 2 Y hayEmpalme = falso; mismoRecurso Y 10:00 < 11:00 Y 11:00 > 10:00	Verdadero; Falso	i ← 2
7	i=2, citas[2]: profesional 3 (mismo), 12:00-13:00; mismoRecurso=verdadero	i <= 2 Y hayEmpalme = falso; mismoRecurso Y 10:00 < 13:00 Y 11:00 > 12:00	Verdadero; Falso (11:00 no es mayor que 12:00)	i ← 3
8	i=3	i <= 2	Falso	Sale del ciclo; hayEmpalme=falso
9	hayEmpalme=falso	Si hayEmpalme (RN-07)	Falso	Continúa
10	folio="CIT-000011", idCita=11 (el número depende de cuántas citas haya)	(sin condición: se guarda la cita en estado pendiente y la bitácora)	—	"Cita agendada. Folio: CIT-000011"

Resultado: cita creada de 10:00 a 11:00 en estado pendiente. Cumple RN-04, RN-05, RN-06 y RN-07.

Algoritmo 2 · Caso de error (empalme de horario)

Datos de entrada: Los mismos datos y citas del caso válido, pero horaInicio=12:30 (horaFin=13:30, dentro del horario 09:00-14:00).

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	estatusPaciente(9)="activo", estatusPersonal(3)="activo"	estatusPaciente = "baja" O estatusPersonal = "baja" (RN-04)	Falso	Continúa

2	**profesionalDaServicio(3,1)=verdadero**	**NO profesionalDaServicio(3,1) (RN-05)**	**Falso**	**Continúa**
3	**duracionServicio(1)=60 min, horaInicio=12:30**	**horaFin ← horaInicio + duración**	**—**	**horaFin = 13:30**
4	**dentroDeHorario(3, 2026-11-09, 12:30, 13:30): el horario es 09:00-14:00**	**NO dentroDeHorario(...) (RN-06)**	**Falso**	**Continúa**
5	**hayEmpalme=falso; i=1**	**—**	**—**	**Inicia el ciclo**
6	**i=1, citas[1]: profesional 4, paciente 2, Consultorio 2; mismoRecurso=falso**	**i <= 2 Y hayEmpalme = falso; mismoRecurso Y ...**	**Verdadero; Falso**	**i ← 2**
7	**i=2, citas[2]: profesional 3 (mismo), 12:00-13:00; mismoRecurso=verdadero**	**i <= 2 Y hayEmpalme = falso; mismoRecurso Y 12:30 < 13:00 Y 13:30 > 12:00**	**Verdadero; Verdadero**	**hayEmpalme ← verdadero; i ← 3**
8	**i=3, hayEmpalme=verdadero**	**i <= 2 Y hayEmpalme = falso**	**Falso**	**Sale del ciclo**
9	**hayEmpalme=verdadero**	**Si hayEmpalme (RN-07)**	**Verdadero**	**"Ese horario ya está ocupado"; Terminar**

Resultado: la cita no se guarda. Se aplica RN-07.

Algoritmo 3 · Caso válido (consulta con una nota nueva y una corrección)

Datos de entrada:

idCita=7),

idProfesional=3 (Terapeuta),

motivoConsulta="Ansiedad y problemas para dormir",

diagnostico="Ansiedad leve",

listaNotas con 2 notas: [1] texto "Nota inicial", corrige=vacío; [2] texto "Corrección de la fecha de inicio de síntomas", corrige=2 (id de una nota anterior).

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	**esPersonalClinico(3)=verdadero (puesto Terapeuta)**	**NO esPersonalClinico(3) (RN-08)**	**Falso**	**Continúa**
2	**estadoCita(7)="confirmada"**	**estadoCita(7) = "cancelada" (RN-08)**	**Falso**	**Continúa**
3	**motivoConsulta="Ansiedad y problemas para dormir"**	**motivoConsulta = ""**	**Falso**	**Continúa**
4	**idConsulta=3; n=1**	**(se guarda la consulta)**	**—**	**Consulta guardada**
5	**n=1, listaNotas[1].corrige=vacío**	**n <= 2; corrige <> vacío**	**Verdadero; Falso**	**Rama Si no: guardarNota(..., vacío); n ← 2**
6	**n=2, listaNotas[2].corrige=2**	**n <= 2; corrige <> vacío (RN-09)**	**Verdadero; Verdadero**	**guardarNota(..., 2) como nota nueva; n ← 3**

7	**n=3**	**n <= 2**	**Falso**	**Sale del ciclo**
8	**estado de la cita ← "completada"**	**(se cambia el estado y se guarda la bitácora)**	**—**	**"Consulta registrada"**

Resultado: consulta y 2 notas guardadas, la nota 2 apunta a la que corrige y no se editó ninguna nota anterior. Cumple RN-08 y RN-09.

Algoritmo 3 · Caso de error (cita cancelada)

Datos de entrada:

idCita=8 (estado "cancelada"),

idProfesional=3 (Terapeuta),

motivoConsulta="Seguimiento",

diagnostico="Sin cambios",

listaNotas con 1 nota.

Paso	Variables y valores	Condición evaluada	Resultado	Salida / acción
1	**esPersonalClinico(3)=verdadero**	**NO esPersonalClinico(3) (RN-08)**	**Falso**	**Continúa**
2	**estadoCita(8)="cancelada"**	**estadoCita(8) = "cancelada" (RN-08)**	**Verdadero**	**"La cita está cancelada"; Terminar**

Resultado: no se guarda consulta ni notas. Se aplica RN-08.

6. Diagramas de flujo

Un diagrama por algoritmo, con la simbología ANSI/ISO 5807: óvalo para inicio o fin, rectángulo para proceso, rombo para decisión, paralelogramo para entrada o salida y flechas para el flujo. Verifica que el diagrama y el pseudocódigo coincidan paso a paso. Sube los archivos fuente a /docs/diseno.

Diagrama del Algoritmo 1

[Imagen del diagrama de flujo]

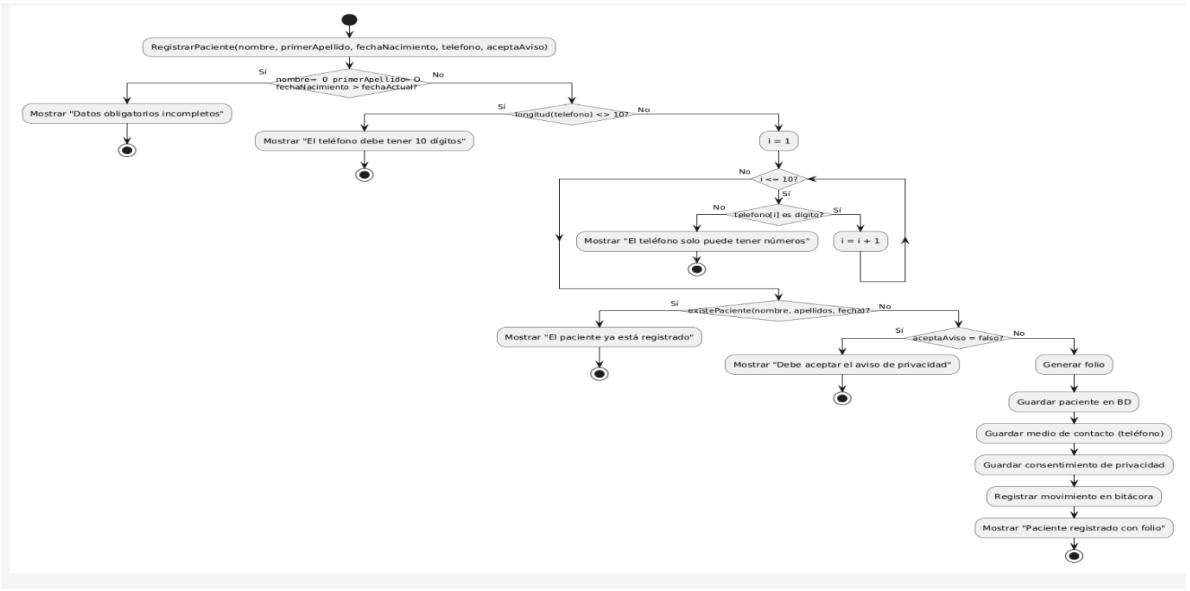


Diagrama del Algoritmo 2

[Imagen del diagrama de flujo]

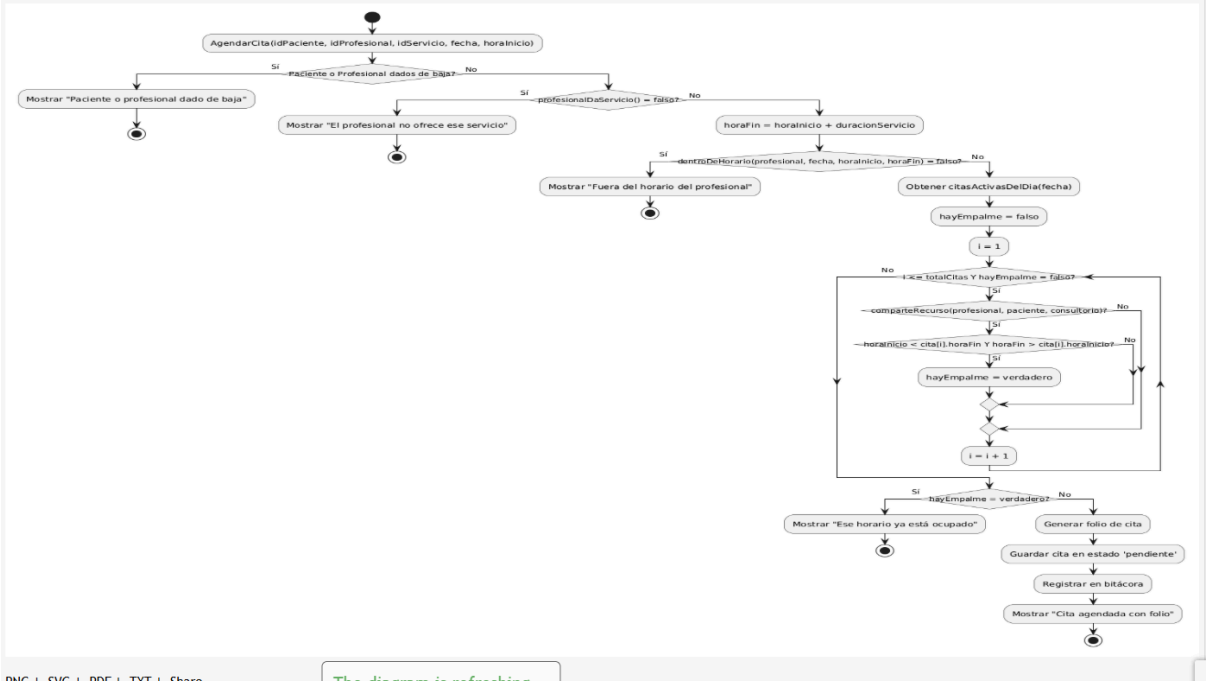
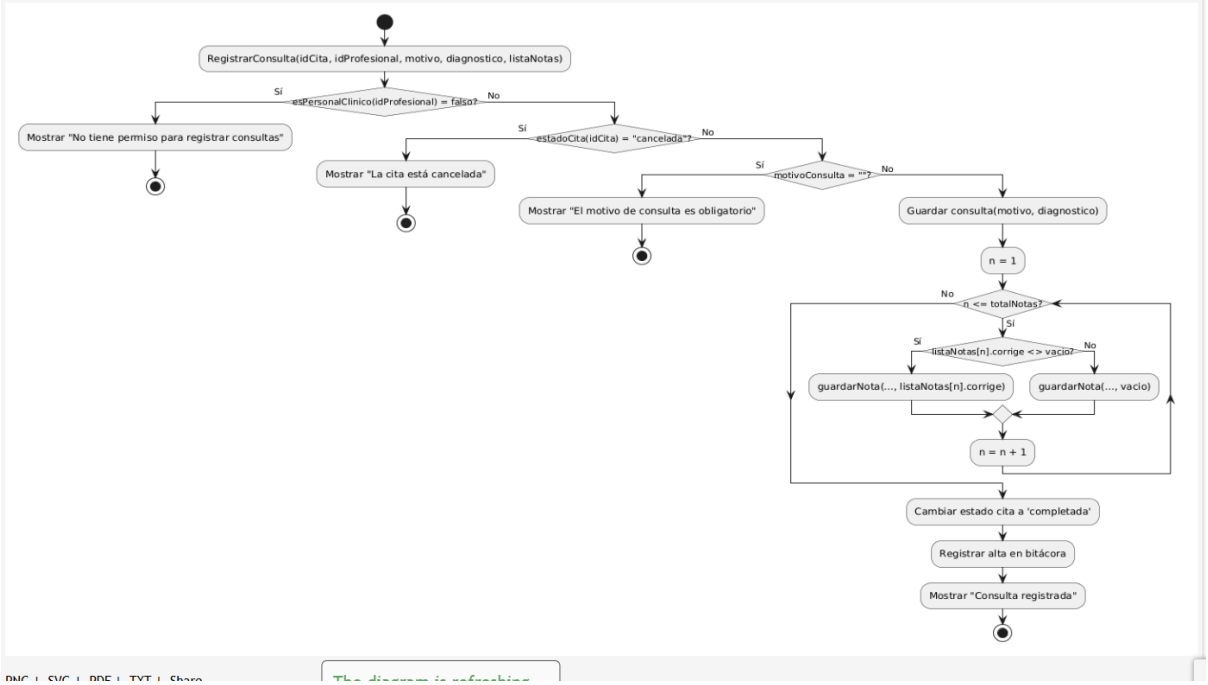


Diagrama del Algoritmo 3

[Imagen del diagrama de flujo]



7. Módulos y principios de diseño funcional

Propone los módulos del sistema y explica cómo aplican la modularidad, la alta cohesión, el bajo acoplamiento y la separación de responsabilidades (interfaz, lógica y datos por separado).

Módulo	Responsabilidad única	Depende de	Principio aplicado y justificación
Vista (Tkinter)	Dibuja la interfaz gráfica, captura los datos del usuario y muestra las alertas. No procesa lógica.	Controlador	Separación de responsabilidades: La interfaz solo se encarga de la presentación. Si mañana cambiamos Tkinter por una página web, el resto del sistema no se rompe.
Controlador	Funciona como orquestador. Capta la petición de la vista, invoca al servicio adecuado y devuelve un resultado.	`services`, `views`	Bajo acoplamiento: Aísla a las pantallas de la lógica dura y de la base de datos, evitando que los botones ejecuten validaciones directas.
services	Contiene toda la lógica de negocio (Algoritmos 1, 2 y 3). Valida reglas,	Repositorio	Alta cohesión: Agrupa todas las funciones lógicas exclusivas del negocio (ej. validaciones de citas) en un solo lugar, sin

	calcula disponibilidad y maneja procesos.		mezclar código de base de datos.
repositorios	Es el único lugar autorizado para comunicarse con PostgreSQL mediante `psycopg2` y ejecutar las sentencias SQL.	Base de datos	Modularidad: Centraliza el acceso a datos. Si una tabla cambia de nombre en PostgreSQL, solo se modifica su archivo repositorio, sin afectar al resto de Python.
models	Define la estructura de los objetos de dominio (Clases puras como `Paciente`, `Cita`, `Personal`).	Ninguno	Separación de responsabilidades: Garantiza que los datos viajen estructurados entre capas usando objetos en lugar de diccionarios sueltos.
Transversales (`utils`, `exceptions`, `config`)	Proveen configuraciones (conexión a BD), herramientas de apoyo y control de errores personalizados.	Ninguno	Reusabilidad: Centraliza el código repetitivo. Un error de base de datos se captura en una excepción personalizada que puede usarse en cualquier capa.

8. Matriz de trazabilidad

Conecta cada requisito funcional con su historia de usuario, su caso de uso y su algoritmo o diagrama. Marca los huecos y explícalos en Observaciones.

Requisito	Historia de usuario	Caso de uso	Algoritmo / diagrama	Observaciones
RF-01	HU-01 (Iniciar sesión) y HU-02 (Roles)	CU-04 (Iniciar sesión)	No se realizo diagrama	El algoritmo de autenticación no se seleccionó entre los 3 prioritarios para diagramar en esta entrega, pero su lógica de contraseñas Hash se implementará directo en la capa de *Services*.

RF-02	HU-03 (Registrar pacientes)	CU-01 (Registrar paciente)	Algoritmo 1 / Diagrama 1	Trazabilidad completa. Valida correctamente las restricciones de duplicidad y longitud del teléfono (RN-01, RN-02 y RN-03).
RF-03	HU-04 (Buscar y editar pacientes)	CU-01 (Como flujo alterno)	No se realizo diagrama	Al ser un flujo de actualización secundario, comparte el mismo repositorio de base de datos que el registro, por lo que se omitió su diagrama para priorizar reglas más complejas.
RF-04	HU-05 (Alta, edición y baja de personal)	CU-05 (Administrar personal)	No se realizo diagrama	Su lógica de inserción es casi idéntica a la de pacientes (CRUD básico). Se decidió no diagramarlo para darle prioridad matemática al agendado de citas.
RF-05	HU-06 (Agendar citas) y HU-08 (Validar horarios)	CU-02 (Agendar cita)	Algoritmo 2 / Diagrama 2	Trazabilidad completa. Resuelve el problema central de la clínica evitando traslapes y validando disponibilidad (RN-04 a RN-07).

RF-11	HU-12 (Registrar notas clínicas)	CU-03 (Registrar consulta)	Algoritmo 3 / Diagrama 3	Trazabilidad completa. Se añadió este requisito para cubrir la restricción normativa de que las notas clínicas originales no se pueden editar ni borrar (RN-08 y RN-09).

9. Revisión entre pares

Intercambien con otro equipo un caso de uso y un algoritmo. Cada equipo busca al menos 2 errores o casos no contemplados en el trabajo del otro; aquí documentan los hallazgos que recibieron y cómo los corrigieron.

Equipo revisor: (número y nombre) · Equipo revisado: (número y nombre)

PENDIENTE DE COMPARTIR REVISION CON OTRO EQUIPO

Hallazgo del otro equipo	Elemento afectado	Corrección aplicada
1.		
2.		
3.		

10. Referencias y declaración de uso de IA

Referencias

- Hernández Sampieri, R. y Mendoza Torres, C. P. (2023). *Metodología de la investigación*. McGraw Hill.
- Clark, W. (2020). *Metodología Ágil: Una guía para principiantes sobre el método y los principios ágiles*. Independently Published.
- draw.io (2026). *Herramienta de diagramación y modelado de software*.
- PlantUML (2026). *Generador de diagramas de actividades y flujos mediante código*.

Declaración de uso de IA

Indiquen la herramienta, para qué la usaron y en qué sección, o escriban "No se utilizó IA". Se permite revisar la notación UML, pedir ejemplos de pseudocódigo de otro dominio y verificar que un diagrama coincida con su algoritmo. No se permite generar los casos de uso o los algoritmos sin partir de sus historias de usuario, ni presentar como propias las pruebas de escritorio hechas por la IA.

Herramienta	Para qué se usó	Sección
Gemini (Google)	Se utilizó como colaborador de apoyo para estructurar el formato de la plantilla, verificar la coherencia lógica entre el pseudocódigo y las restricciones de la base de datos PostgreSQL, y redactar las justificaciones técnicas de los principios de diseño modular.	Secciones 4, 7 y 8.
Secciones 4, 7 y 8.	Generación y modelado visual de los diagramas de flujo estándar ANSI/ISO 5807.	Sección 6.